

Date:
Student Name:
Student ID:

AI Workshop #5

Detecting Patterns with Unsupervised Learning

5.1. Introduction

In this workshop, we are going to learn about unsupervised learning and how to use it in real-world situations. By the end of this workshop, you will have a better understanding of the following topics:

By the end of this workshop, you will have a better understanding of these topics:

- Unsupervised learning definition
- Clustering data with the K-Means algorithm
- Estimating the number of clusters with the Mean Shift algorithm
- Estimating the quality of clustering with silhouette scores
- Gaussian Mixture Models
- Building a classifier based on Gaussian Mixture Models
- Finding subgroups in stock markets the using Affinity Propagation model
- Segmenting the market based on shopping patterns

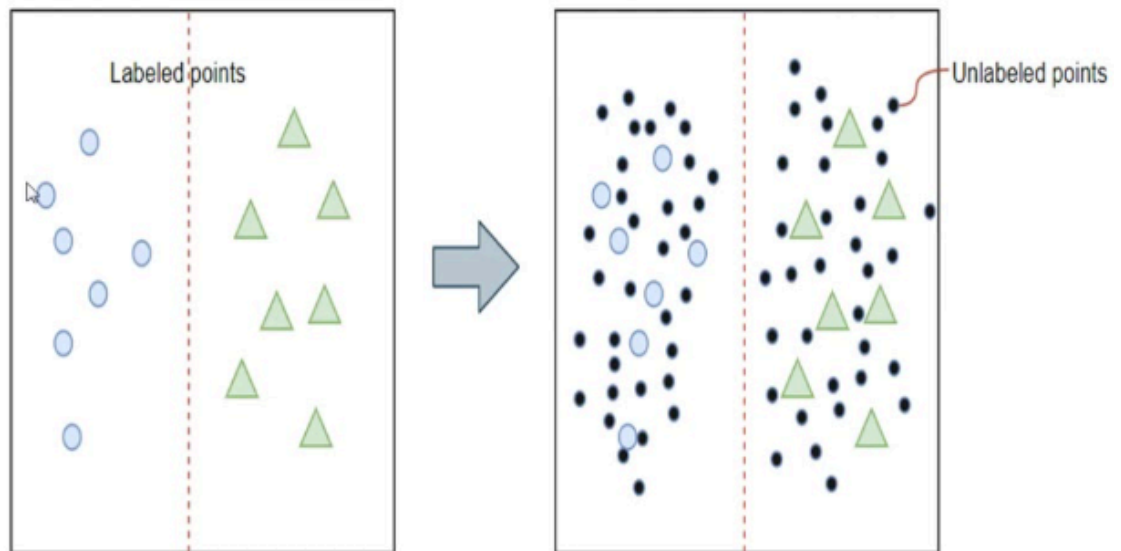
5.2 What is unsupervised learning?

Unsupervised Learning (UL) refers to the process of building machine learning models without using labeled training data.

Unsupervised learning finds applications in diverse fields of study, including market segmentation, stock markets, natural language processing, and computer vision, to name a few.

In the previous workshops, we were dealing with data that had labels associated with it. When we have labeled training data, algorithms learn to classify data based on those labels. In the real world, labeled data might not always be available.

labeled data and unlabeled data



Sometimes, a large quantity of data exists without labeling and it needs to be categorized in some way. This is the perfect use case for unsupervised learning. Unsupervised learning

5.3 Clustering data with the K-Means algorithm

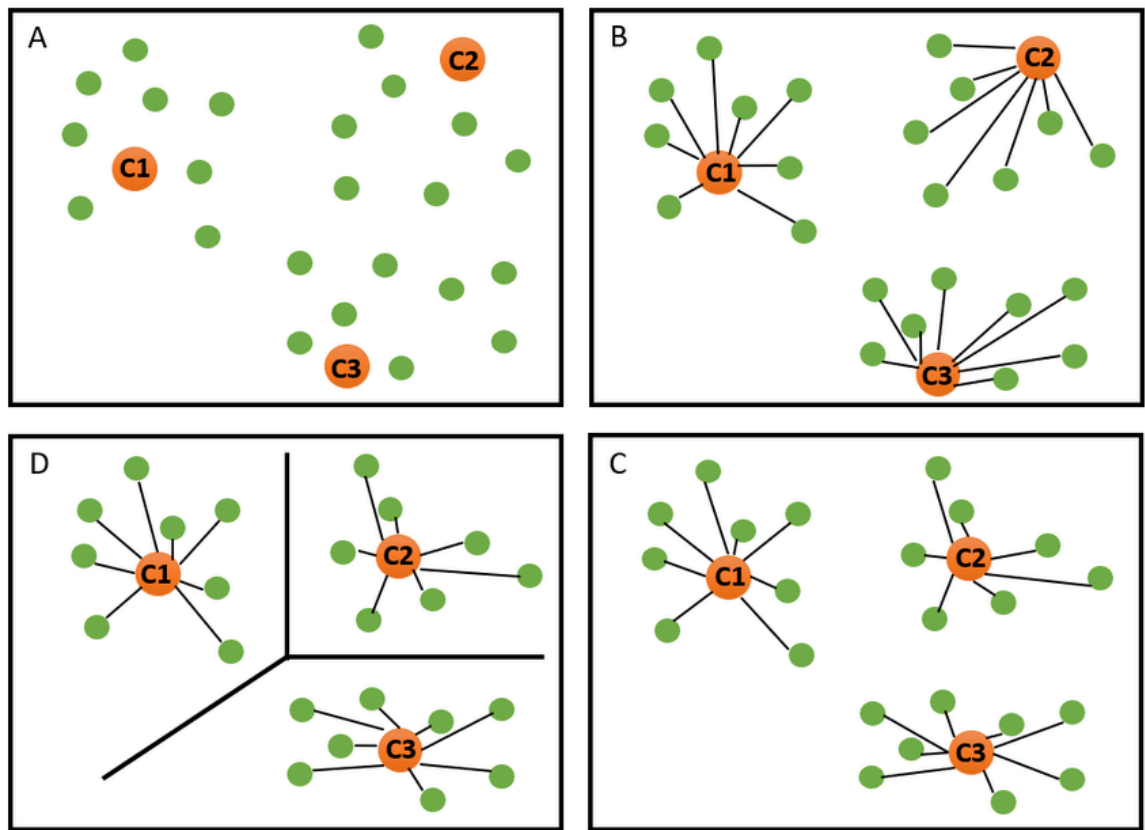
Clustering is one of the most popular unsupervised learning techniques. This technique is used to analyze data and find clusters within that data. In order to find these clusters, we use a similarity measurement such as the Euclidean distance to find subgroups. This similarity measure can estimate the tightness of a cluster. Clustering is the process of organizing data into subgroups whose elements are like each other.

The goal of the algorithm is to identify the intrinsic properties of data points that make them belong to the same subgroup. There is no universal similarity metric that works in all cases. For example, we might be interested in finding the representative data point for each subgroup, or we might be interested in finding the outliers in the data. Depending on the situation, different metrics might be more appropriate than others.

The **K-Means algorithm** is a well-known algorithm for clustering data. In order to use it, the number of clusters is assumed beforehand. The data is segmented into K subgroups using various data attributes. The number of clusters is fixed, and the data is classified based on that number. The main idea here is that we need to update the locations of the centroids with each iteration. A centroid is the location representing the center of the cluster. We continue iterating until we have placed the centroids at their optimal locations.

We can see that the initial placement of centroids plays an important role in the algorithm. These centroids should be placed in a clever manner, because this directly impacts the results. A good strategy is to place them as far away from each other as possible.

How K-means work



The basic K-Means algorithm places these centroids randomly where K-Means++ chooses these points algorithmically from the input list of data points. It tries to place the initial centroids far from each other so that they converge quickly. We then go through the training dataset and assign each data point to the closest centroid.

Once we go through the entire dataset, the first iteration is over. The points have been grouped based on the initialized centroids. The location of the centroids is recalculated based on the new clusters that were obtained at the end of the first iteration. Once a new set of K centroids is obtained, the process is repeated. We iterate through the dataset and assign each point to the closest centroid.

As the steps keep on getting repeated, the centroids keep moving to their equilibrium position. After a certain number of iterations, the centroids do not change their locations

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn import metrics
```

2. Load the input data from the file:

```
# Load input data
X = np.loadtxt('data_clustering.txt', delimiter=',')
```

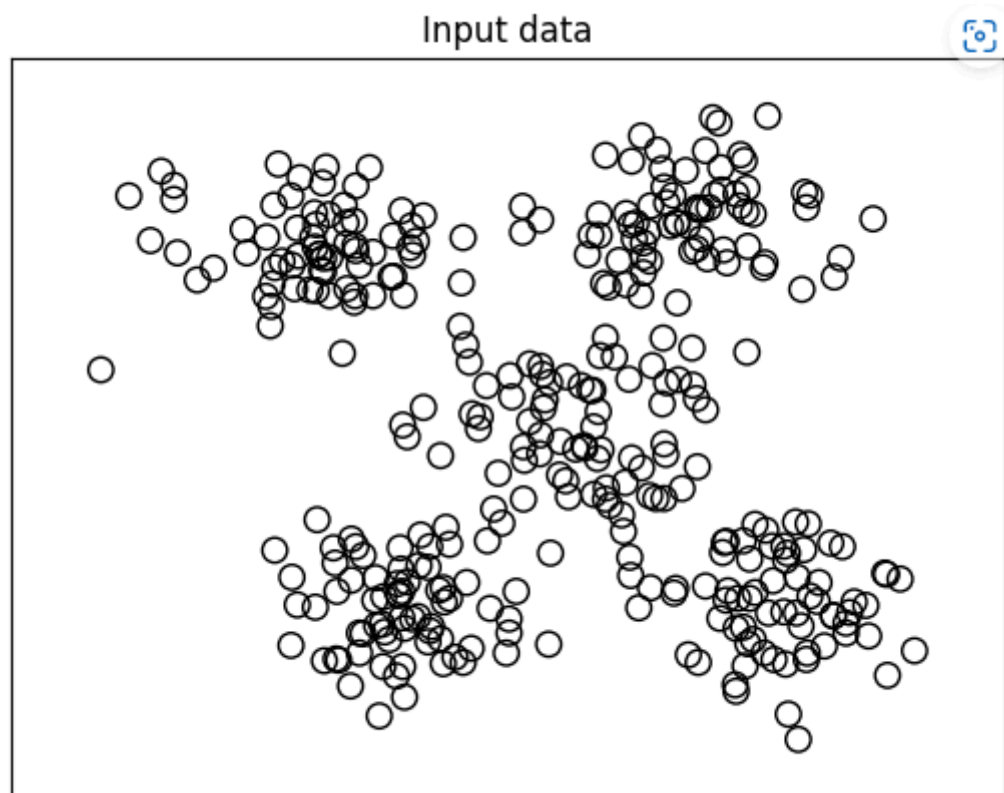
3. Define the number of clusters before applying the K-Means algorithm:

```
num_clusters = 5
```

4. Visualize the input data to see what the spread looks like:

```
# Plot input data
plt.figure()
plt.scatter(X[:,0], X[:,1], marker='o', facecolors='none',
            edgcolors='black', s=80)
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
plt.title('Input data')
plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)
plt.xticks(())
plt.yticks(())
```

5. If everything fine, you should see this.



6. Combine all the python code and see whether you got this figure.

In [1]: *## Try your code in here*



It can be seen that there are five groups within this data. Create the KMeans object using the initialization parameters. The init parameter represents the method of initialization to select the initial centers of clusters.

6. Instead of selecting them randomly, we use **k-means++** to select these centers in a smarter way. This ensures that the algorithm converges quickly. The `n_clusters` parameter refers to the number of clusters. The `n_init` parameter refers to the number of times the algorithm should run before deciding upon the best outcome:

```
# Create KMeans object
kmeans = KMeans(init='k-means++', n_clusters=num_clusters, n_init=10)
```

7. Train the K-Means model with the input data:

```
# Train the KMeans clustering model
kmeans.fit(X)
```

8. To visualize the boundaries, we need to create a grid of points and evaluate the model on all those points. Let's define the step size of this grid:

```
# Step size of the mesh
step_size = 0.01
```

9. We define the grid of points and ensure that we are covering all the values in the input data:

```
# Define the grid of points to plot the boundaries
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
x_vals, y_vals = np.meshgrid(np.arange(x_min, x_max, step_size),
                             np.arange(y_min, y_max, step_size))
```

10. Predict the outputs for all the points on the grid using the trained K-Means model:

```
# Predict output labels for all the points on the grid
output = kmeans.predict(np.c_[x_vals.ravel(), y_vals.ravel()])
```

11. Plot all output values and color each region:

```
# Plot different regions and color them
output = output.reshape(x_vals.shape)
plt.figure()
plt.clf()
plt.imshow(output, interpolation='nearest',
           extent=(x_vals.min(), x_vals.max(),
                  y_vals.min(), y_vals.max()),
           cmap=plt.cm.Paired,
           aspect='auto',
           origin='lower')
```

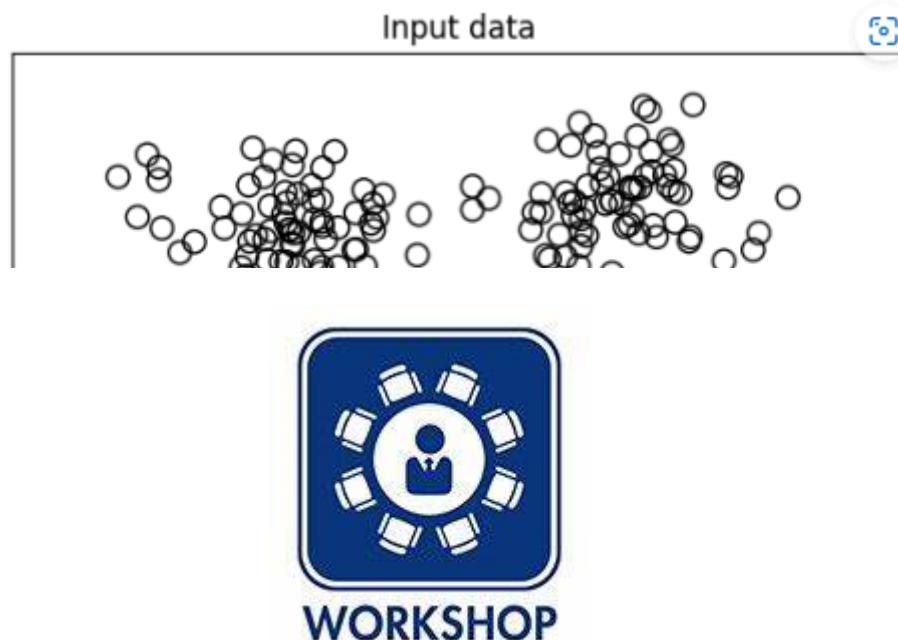
12. Overlay input data points on top of these colored regions:

```
# Overlay input points
plt.scatter(X[:,0], X[:,1], marker='o', facecolors='none',
edgecolors='black', s=80)
```

13. Plot the centers of the clusters obtained using the K-Means algorithm:

```
# Plot the centers of clusters
cluster_centers = kmeans.cluster_centers_
plt.scatter(cluster_centers[:,0], cluster_centers[:,1],
marker='o', s=210, linewidths=4, color='black',
zorder=12, facecolors='black')
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
plt.title('Boundaries of clusters')
plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)
plt.xticks(())
plt.yticks(())
plt.show()
```

14. If everything is okay, you should see this K-mean clustering result figures.



Workshop 5.1 : K-means Clustering

1. Combine all the above code and named it as **kmeans.py**.
2. If everything is okay, you should see the above 2 figures. The first screenshot is the input data. The second screenshot represents the boundaries obtained using K-Means.
3. Save and submit the source code and these figures.
4. What are the black-filled circle in each cluster?
5. Set the cluster number into i) 3, ii) 4, iii) 7, iv) 9 and discuss what do you find?
6. Which cluster number do you think is the best to describe these points? Why?

5.4 Estimating the number of clusters with the Mean Shift algorithm

Mean Shift is a powerful algorithm used in unsupervised learning. It is a nonparametric algorithm used frequently for clustering.

It is non-parametric because it does not make any assumptions about the underlying distributions. This contrasts with parametric techniques, where it is assumed that the underlying data follows a standard probability distribution. Mean Shift finds a lot of applications in fields such as object tracking and real-time data analysis.

In the Mean Shift algorithm, the whole feature space is considered as a probability density function. We start with the training dataset and assume that it has been sampled from a probability density function.

In this framework, the clusters correspond to the local maxima of the underlying distribution. If there are K clusters, then there are K peaks in the underlying data distribution and Mean Shift will identify those peaks.

The goal of Mean Shift is to identify the location of centroids. For each data point in the training dataset, it defines a window around it. It then computes the centroid for this window and updates the location to this new centroid. It then repeats the process for this new

location by defining a window around it. As we keep doing this, we move closer to the peak of the cluster. Each data point will move towards the cluster it belongs to. The movement is towards a region of higher density.

The centroids (also called means) keep on getting shifted towards the peaks of each cluster. The algorithm gets its name from the fact that the means keep getting shifted. The shift continues to happen until the algorithm converges, at which stage the centroids don't move anymore.

Let's see how to use MeanShift to estimate the optimal number of clusters in the given

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import MeanShift, estimate_bandwidth
from itertools import cycle
```

2. Load the input data:

```
# Load data from input file
X = np.loadtxt('data_clustering.txt', delimiter=',')
```



Estimate the bandwidth of the input data. **Bandwidth** is a parameter of the underlying kernel density estimation process used in the Mean Shift algorithm. The bandwidth affects the overall convergence rate of the algorithm and the number of clusters that we will end up with in the end. Hence, this is a crucial parameter, if the bandwidth is too small, it might result in too many clusters, whereas if the value is too large, then it will merge distinct clusters.

3. The quantile parameter impacts how the bandwidth is estimated. A higher value for a quantile will increase the estimated bandwidth, resulting in fewer clusters:

```
# Estimate the bandwidth of X
bandwidth_X = estimate_bandwidth(X, quantile=0.1, n_samples=len(X))
```

4. Then train the Mean Shift clustering model using the estimated bandwidth:

```
# Cluster data with MeanShift
meanshift_model = MeanShift(bandwidth=bandwidth_X, bin_seeding=True)
meanshift_model.fit(X)
```

5. Extract the centers of all the clusters:

```
# Extract the centers of clusters
cluster_centers = meanshift_model.cluster_centers_
```

```
print('\nCenters of clusters:\n', cluster_centers)
```

6. Extract the number of clusters:

```
# Estimate the number of clusters
labels = meanshift_model.labels_
num_clusters = len(np.unique(labels))
print("\nNumber of clusters in input data =", num_clusters)
```

7. Visualize the data points:

```
# Plot the points and cluster centers
plt.figure()
markers = 'o*xvs'
for i, marker in zip(range(num_clusters), markers):
    # Plot points that belong to the current cluster
    plt.scatter(X[labels==i, 0], X[labels==i, 1], marker=marker,
                color='black')
```

8. Plot the center of the current cluster:

```
# Plot the cluster center
cluster_center = cluster_centers[i]
plt.plot(cluster_center[0], cluster_center[1], marker='o',
         markerfacecolor='black', markeredgecolor='black',
         markersize=15)
plt.title('Clusters')
plt.show()
```

9. If everything go well, you will see the results like this:

```
Centers of clusters:  
[[2.95568966 1.95775862]  
 [7.20690909 2.20836364]  
 [2.17603774 8.03283019]  
 [5.97960784 8.39078431]  
 [4.99466667 4.65844444]]
```

```
Number of clusters in input data = 5
```



Workshop 5.2 : Estimate K-means Cluster

1. Combine all the above code and named it as **mean_shift.py**.
2. If everything is okay, you should see the above figures with the resulted cluster number and their centroid. It also shows the clustering results.
3. Save and submit the source code and these figures.
4. Add 100 random number into this dataset and redo the calculation again. What is your findings?
5. Save and submit the new result figures.



This completes our overview of Mean Shift.

So far, we've dealt with how to cluster out data. Next, we'll move on to how we can estimate the quality of clustering, using the **silhouette method**.

5.5 Estimating the quality of clustering with silhouette scores

If data is naturally organized into several distinct clusters, then it is easy to visually examine it and draw some inferences. This is rarely the case in the real world, unfortunately. Data in the real world is huge and messy. So, we need a way to quantify the quality of the clustering.

Silhouette refers to a method used to check the consistency of clusters in data. It gives an estimate of how well each data point fits with its cluster. The silhouette score is a metric that measures the similarity of a data point to its own cluster, as compared to other clusters. The silhouette score works with any similarity metric.

For each data point, the silhouette score is computed using the following formula:

$$\text{silhouette score} = (p - q) / \max(p, q)$$

Here, p is the mean distance to the points in the nearest cluster that the data point is not a part of, and q is the mean intra-cluster distance to all the points in its own cluster.

The value of the silhouette score range lies between -1 and 1. A score closer to 1 indicates that the data point is very similar to other data points in the cluster, whereas a score closer to -1 indicates that the data point is not like other data points in the cluster. One way to think about it is if there are too many points with negative silhouette scores, then there may be too few or too many clusters in the data. We need to run the clustering algorithm again to find the optimal number of clusters.

Ideally, we want to have a high positive value. Depending on the business problem, we do not need to optimize and have the highest possible value, but in general, if we have a silhouette score that is close to 1, it indicates that the data clustered nicely. If the scores are close to -1, it indicates that the variable that we are using to classify is noisy and does not contain much of a signal.

.....

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import metrics
from sklearn.cluster import KMeans
```

2. We will be using the data in the **data_quality.txt** file provided to you. Each line contains two comma-separated numbers:

```
# Load data from input file
X = np.loadtxt('data_quality.txt', delimiter=',')
```

3. Plot input data:

```
# Plot input data
plt.figure()
plt.scatter(X[:,0], X[:,1], color='black', s=80, marker='o',
facecolors='none')
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
plt.title('Input data')
plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)
plt.xticks(())
plt.yticks(())
```

4. Initialize the variables. The values array will contain a list of values to iterate over and to find the optimal number of clusters:

```
# Initialize variables
scores = []
```

```
values = np.arange(2, 10)
```

5. Iterate through all the values and build a K-Means model during each iteration:

```
# Iterate through the defined range
for num_clusters in values:
    # Train the KMeans clustering model
    kmeans = KMeans(init='k-means++', n_clusters=num_clusters, n_init=10)
    kmeans.fit(X)
```

6. Estimate the silhouette score for the current clustering model using the Euclidean distance metric:

```
score = metrics.silhouette_score(X, kmeans.labels_,
                                  metric='euclidean', sample_size=len(X))
```

7. Print the silhouette score for the current value:

```
print("\nNumber of clusters =", num_clusters)
print("Silhouette score =", score)

scores.append(score)
```

8. Visualize the silhouette scores for various values:

```
# Plot silhouette scores
plt.figure()
plt.bar(values, scores, width=0.7, color='black', align='center')
plt.title('Silhouette score vs number of clusters')
```

9. Extract the best score and the corresponding value for the number of clusters:

```
# Extract best score and optimal number of clusters
num_clusters = np.argmax(scores) + values[0]
print('\nOptimal number of clusters =', num_clusters)
```

10. Display the results, you should see the following 3 figures.

```
plt.show()
```

Number of clusters = 2
Silhouette score = 0.47762624870454473

Number of clusters = 3
Silhouette score = 0.5471742411734872

Number of clusters = 4
Silhouette score = 0.579480188968759

Number of clusters = 5
Silhouette score = 0.5890032635647954

Number of clusters = 6
Silhouette score = 0.6096904118954452

Number of clusters = 7
Silhouette score = 0.5531506134707689

Number of clusters = 8
Silhouette score = 0.4967137698762458

Number of clusters = 9
Silhouette score = 0.46666773295041014

Optimal number of clusters = 6



The contains:

1. The calculation results of all different clusters and their Silhouette scores. The optimal is 6.
2. The visualization of the dataset.
3. The Silhouette score vs number of clusters figure chart with the peak of Cluster no 6 with score around 0.6.



Workshop 5.3 : Estimation of Clustering with Silhouette Scores

1. Combine all the above code and named it as **silhouette_score.py**.
2. If everything is okay, you should see the above figures with the resulted cluster number and their silhouette score values, both chart and table formats.
3. Save and submit the source code and these figures.
4. Add 100 random number into this dataset and redo the calculation again. What is your findings?
5. Save and submit the new result figures.



5.6 Gaussian Mixture Models

Before we discuss **Gaussian Mixture Models (GMMs)**, let's first understand what a Mixture Model is. A Mixture Model is a type of probability density model where it is assumed that the data is governed by several component distributions. If these distributions are Gaussian, then the model becomes a Gaussian Mixture Model. These component distributions are combined in order to provide a multi-modal density function, which becomes a mixture model.

Let's look at an example to understand how **Mixture Models** work. We want to model the shopping habits of all the people in South America. One way to do it would be to model the whole continent and fit everything into a single model, but people in different countries shop differently. We therefore need to understand how people in individual countries shop and how they behave.

To get a good representative model, we need to account for all the variations within the continent. In this case, we can use mixture models to model the shopping habits of individual countries and then combine all of them into a Mixture Model.

This way, nuances in the data of the underlying behavior of individual countries are not missed. By not enforcing a single model on all of the countries, a more accurate model is created.

An interesting point to note is that mixture models are semi-parametric, which means that they are partially dependent on a set of predefined functions. They can provide greater precision and flexibility in modeling the underlying distributions of the data. They can smooth the gaps that result from having sparse data.

Once the function is defined, the mixture model goes from being semi-parametric to parametric. Hence a GMM is a parametric model represented as a weighted summation of component Gaussian functions. We assume that the data is being generated by a set of Gaussian models that are combined in some way. GMMs are very powerful and are used in many fields. The parameters of the GMM are estimated from training data using algorithms like **Expectation-Maximization (EM)** or **Maximum A-Posteriori (MAP) estimation**. Some of the popular applications of GMM include image database retrieval, modeling stock market fluctuations, biometric verification, and so on.

Building a classifier based on Gaussian Mixture Models

1. Let's build a classifier based on a Gaussian Mixture Model. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import patches
from sklearn import datasets
from sklearn.mixture import GaussianMixture
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import train_test_split
```

2. Let's use the Iris dataset available in scikit-learn for analysis:

```
# Load the iris dataset
iris = datasets.load_iris()

X, y = datasets.load_iris(return_X_y=True)
```

3. Split the dataset into training and testing using an 80/20 split. The `n_splits` parameter specifies the number of subsets you'll obtain. We are using a value of 5, which means the dataset will be split into five parts. We will use four parts for training and one part for testing, which gives a split of 80/20:

```
# Split dataset into training and testing (80/20 split)
skf = StratifiedKFold(n_splits=5) #
skf.get_n_splits(X, y)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4,
random_state=0)
```

4. Extract the number of classes in the training data:

```
# Extract the number of classes
num_classes = len(np.unique(y_train))
```

5. Build a GMM-based classifier using the relevant parameters. The `n_components` parameter specifies the number of components in the underlying distribution. In this case, it will be the number of distinct classes in the data. We need to specify the type of covariance to use. In this case, full covariance will be used. The `init_params` parameter controls the parameters that need to be updated during the training process. The value `kmeans` was used, which means weights and covariance parameters will be updated during training. The `max_iter` parameter refers to the number of Expectation-Maximization iterations that will be performed during training:

```
# Build GMM
classifier = GaussianMixture(n_components=num_classes,
covariance_type='full',
init_params='kmeans', max_iter=20)
```

6. Initialize the means of the classifier:

```
# Initialize the GMM means
classifier.means_ = np.array([X_train[y_train == i].mean(axis=0)
for i in range(num_classes)])
```

7. Train the Gaussian mixture model classifier using the training data:

```
# Train the GMM classifier
classifier.fit(X_train)
```

9. Visualize the boundaries of the classifier. Then extract the eigenvalues and eigenvectors to estimate how to draw the elliptical boundaries around the clusters. Let's go ahead and plot the following:

```
# Draw boundaries
plt.figure()
colors = 'bgr'
for i, color in enumerate(colors):
    # Extract eigenvalues and eigenvectors
    eigenvalues, eigenvectors = np.linalg.eigh(
        classifier.covariances_[i][:2, :2])
```

10. Normalize the first eigenvector:

```
# Normalize the first eigenvector
norm_vec = eigenvectors[0] / np.linalg.norm(eigenvectors[0])
```

11. The ellipses need to be rotated to accurately show the distribution. Estimate the angle:

```
# Extract the angle of tilt
angle = np.arctan2(norm_vec[1], norm_vec[0])
angle = 180 * angle / np.pi
```

12. Magnify the ellipses for visualization. The eigenvalues control the size of the ellipses:

```
# Scaling factor to magnify the ellipses
# (random value chosen to suit our needs)
scaling_factor = 8
eigenvalues *= scaling_factor
```

13. Draw the ellipse:

```
# Draw the ellipse
ellipse = patches.Ellipse(classifier.means_[i, :2],
                          eigenvalues[0], eigenvalues[1], angle=180 +
angle,
                          color=color)
axis_handle = plt.subplot(1, 1, 1)
ellipse.set_clip_box(axis_handle.bbox)
ellipse.set_alpha(0.6)
axis_handle.add_artist(ellipse)
```

14. Overlay input data on the figure:

```
# Plot the data
colors = 'bgr'
for i, color in enumerate(colors):
    cur_data = iris.data[iris.target == i]
    plt.scatter(cur_data[:,0], cur_data[:,1], marker='o',
                facecolors='none', edgecolors='black', s=40,
                label=iris.target_names[i])
```

15. Overlay test data on this figure:

```
test_data = X_test[y_test == i]
plt.scatter(test_data[:,0], test_data[:,1], marker='s',
            facecolors='black', edgecolors='black', s=40
            ,label=iris.target_names[i])
```

16. Compute the predicted output for the training and testing data:

```
# Compute predictions for training and testing data
y_train_pred = classifier.predict(X_train)
accuracy_training = np.mean(y_train_pred.ravel() == y_train.ravel()) * 100
```

```

print('Accuracy on training data =', accuracy_training)

y_test_pred = classifier.predict(X_test)
accuracy_testing = np.mean(y_test_pred.ravel() == y_test.ravel()) * 100
print('Accuracy on testing data =', accuracy_testing)

plt.title('GMM classifier')
plt.xticks(())
plt.yticks(())

plt.show()

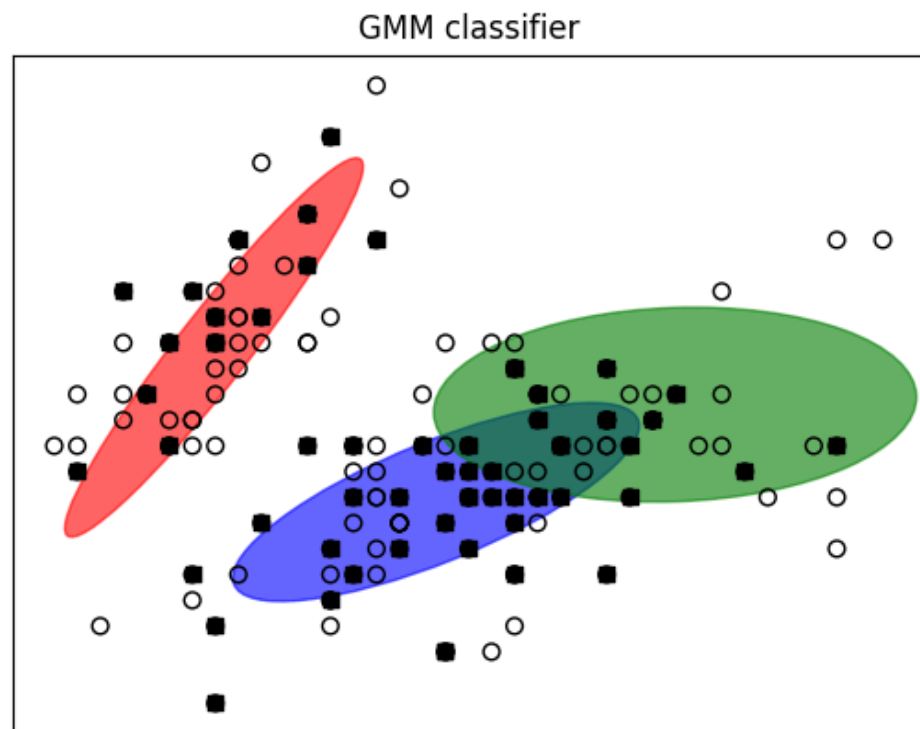
```

17. If everything goes well, you should got this nice figures, together with the Accuracy on training data and testing data.

```

Accuracy on training data = 87.5
Accuracy on testing data = 86.6666666667

```



The input data consists of three distributions. The three ellipses of various sizes and angles represent the underlying distributions in the input data.



Workshop 5.4 : Gaussian Mixture Models

1. Combine all the above code and named it as **gmm_classifier.py**.
2. If everything is okay, you should see the above figures with 3 clusters, together with the overall accuracy of training/test data.
3. Save and submit the source code and this figure.
4. Try to run this program several time, you will see there are some difference. What is that and why it happens.
5. How can we fix this problem?



5.7 Mini-project: Market Segmentation based on Shopping Patterns

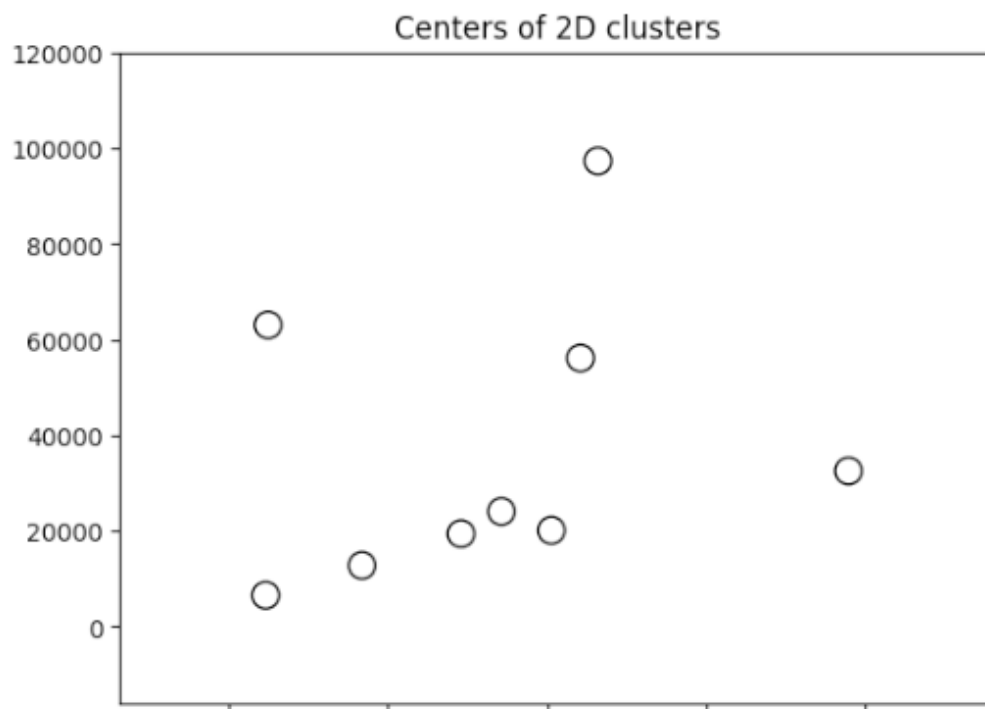
Let's apply unsupervised learning techniques to segment the market based on customer shopping habits.

1. You have been provided with a file named **sales.csv**. This file contains the sales details of a variety of tops from several retail clothing stores.
2. The goal is to identify the patterns and segment the market based on the number of units sold in those stores.
3. By using the **Mean Shift Algorithm** being learnt in this workshop, try to write a Python program to analyze and segment customer shopping habits.
4. If you market segmentation program works, you should see something like this.
5. Save your program as **market_segmentation.py** and submit both your program and result table figures.

Number of clusters in input data = 9

Centers of clusters:

Tsh	Tan	Hal	Tur	Tub	Swe
9823	4637	6539	2607	2228	1239
38589	44199	56158	5030	24674	4125
104972	29186	19415	16016	5060	9372
38741	40539	20120	35059	255	50710
35314	16745	12775	66900	1298	5613
28333	34263	24065	5575	4229	18076
22617	77873	32543	1005	21035	837
14987	46397	97393	1127	37315	3235
7852	4939	63081	134	40066	1332



5.8 Summary

In this workshop, we started by discussing unsupervised learning and its applications. We then learned about clustering and how to cluster data using the K-Means algorithm.

We discussed how to estimate the number of clusters with the Mean Shift algorithm. We talked about silhouette scores and how to estimate the quality of clustering.

We learned about Gaussian Mixture Models and how to build a classifier based on them.

We also discussed the Affinity Propagation model and used it to find subgroups within the stock market.

We then applied the Mean Shift algorithm to segment the market based on shopping patterns.